

LAPORAN PROYEK AKHIR

Parallel K-Means Image Segmentation Benchmark

Sequential | OpenMP | OpenCL



Kelompok Benchmark Pari-Pari

Adhistya Wildan Maulana Akbar :	25032014015
Habib Farah Adi Maskur :	25032014034
Michael Frearjuna :	25032014001

**KECERDASAN ARTIFISIAL
FAKULTAS MATEMATIKA DAN ILMU PENGETAHUAN ALAM**

1. Deskripsi Program

Program `kmeans_benchmark` adalah aplikasi C++ yang mengimplementasikan algoritma K-Means Clustering untuk segmentasi citra (image segmentation). Program ini dirancang untuk membandingkan performa tiga pendekatan komputasi yang berbeda: Sequential (CPU single-thread), OpenMP (CPU multi-thread), dan OpenCL (GPU/accelerator).

Keistimewaan utama program ini adalah penggunaan jarak Mahalanobis sebagai metrik jarak antar piksel, berbeda dengan K-Means konvensional yang menggunakan jarak Euclidean. Jarak Mahalanobis memperhitungkan korelasi antar channel warna (R, G, B), sehingga menghasilkan kluster yang lebih akurat secara statistik.

Fitur Utama

- Segmentasi citra ke dalam $K = 5$ cluster warna
- Jarak Mahalanobis dengan invers covariance matrix 3×3
- Tiga varian implementasi: Sequential, OpenMP, OpenCL
- Benchmark otomatis dengan pengukuran waktu (ms) dan speedup
- Output gambar segmentasi untuk setiap metode

Teknologi yang Digunakan

Komponen	Teknologi	Keterangan
Bahasa	C++17	Standar modern dengan STL
Image Library	OpenCV 4	Load, resize, blur, simpan gambar
Paralelisme CPU	OpenMP	<code>#pragma omp parallel for</code>
Paralelisme GPU	OpenCL	Kernel <code>assign_labels</code> di GPU
Build System	Makefile (g++)	Kompilasi dengan <code>-O2 -fopenmp</code>

2. Latar Belakang

Segmentasi citra adalah salah satu tugas fundamental dalam pengolahan citra dan visi komputer. Proses ini mempartisi piksel-piksel suatu gambar ke dalam kelompok-kelompok (cluster) berdasarkan kemiripan warnanya, sehingga menghasilkan representasi yang lebih sederhana dari citra asli.

Algoritma K-Means merupakan salah satu metode clustering yang paling banyak digunakan. Namun pada citra beresolusi tinggi, K-Means memiliki kompleksitas $O(N \times K \times I)$ di mana N adalah jumlah piksel, K adalah jumlah cluster, dan I adalah jumlah iterasi, sehingga menjadi bottleneck komputasi yang signifikan.

Komputasi paralel menawarkan solusi untuk mengatasi tantangan performa ini. OpenMP memungkinkan paralelisasi pada CPU menggunakan multiple thread, sedangkan OpenCL memungkinkan offloading komputasi ke GPU yang memiliki ribuan core paralel. Perbandingan keduanya memberikan wawasan tentang tradeoff antara kemudahan implementasi (OpenMP) dan potensi speedup maksimal (OpenCL).

Proyek ini mengambil studi kasus segmentasi citra sawah (sawah6.png). Penggunaan jarak Mahalanobis dipilih karena mampu memodelkan korelasi antar channel RGB secara lebih tepat dibandingkan Euclidean distance.

3. Cara Kerja Program

Program berjalan melalui lima tahap utama, dengan tiga implementasi K-Means yang berjalan secara bergantian untuk keperluan benchmark:

3.1 Load dan Preprocess Citra

Program membaca file gambar menggunakan OpenCV (`cv::imread`), kemudian melakukan tiga operasi preprocessing:

- Resize ke ukuran `targetSize x targetSize` menggunakan algoritma LANCZOS4
- Gaussian blur dengan `sigma = 0.5` untuk meredam noise halus pada citra
- Konversi color space dari BGR (format default OpenCV) ke RGB

Hasil disimpan sebagai array float 1D berukuran $N \times 3$, di mana $N = \text{width} \times \text{height}$ adalah total piksel.

3.2 Hitung Covariance Matrix dan Inversnya

Program menghitung covariance matrix 3×3 dari seluruh piksel untuk menangkap korelasi antar channel RGB. Regularisasi diagonal sebesar $1e-5$ ditambahkan untuk mencegah matriks singular. Kemudian matriks diinverskan menggunakan metode cofactor (adjugate) untuk mendapatkan `invCov` yang digunakan dalam perhitungan jarak Mahalanobis.

3.3 Inisialisasi Centroid

$K = 5$ centroid awal dipilih secara acak dari piksel-piksel gambar menggunakan seed deterministik (`seed = 42`) agar hasil dapat direproduksi. Ketiga implementasi menggunakan centroid awal yang identik untuk memastikan perbandingan yang adil.

3.4 Iterasi K-Means (3 Implementasi)

Setiap implementasi menjalankan loop iterasi hingga konvergensi atau maksimum 300 iterasi. Setiap iterasi terdiri dari tiga langkah:

Langkah 1 - Assignment:

Setiap piksel dihitung jaraknya ke semua K centroid menggunakan jarak Mahalanobis: $d(x, c) = \sqrt{(x - c)^T \cdot \Sigma^{-1} \cdot (x - c)}$, lalu diberikan label centroid terdekat.

Langkah 2 - Update Centroid:

Setiap centroid diperbarui sebagai rata-rata (mean) dari semua piksel yang ditugaskan ke cluster tersebut.

Langkah 3 - Cek Konvergensi:

Iterasi berhenti jika selisih absolut setiap elemen centroid antara iterasi kini dan sebelumnya kurang dari $1e-6$.

Perbedaan Implementasi Ketiga Varian

Aspek	Sequential	OpenMP	OpenCL
Assignment	1 thread, loop N piksel	#pragma omp parallel for	GPU kernel: 1 work-item/piksel
Update	Akumulasi global langsung	Thread-local + critical section	Download labels, update di CPU
Hardware	CPU (1 core)	CPU (semua core)	GPU / accelerator

3.5 Simpan Hasil dan Print Benchmark

Setelah tiap implementasi selesai, label piksel digunakan untuk membuat gambar hasil segmentasi: setiap piksel diwarnai dengan warna centroid-nya. Program mencetak tabel benchmark berisi waktu eksekusi (ms) dan speedup relatif terhadap implementasi sequential.

4. Cara Simulasi Program

4.1 Persyaratan Sistem

- Compiler: g++ dengan dukungan C++17 dan OpenMP (-fopenmp)
- Library: OpenCV 4 (pkg-config tersedia)
- Library: OpenCL SDK (libOpenCL, driver GPU atau CPU fallback)
- OS: Linux/Unix (Makefile menggunakan pkg-config)

4.2 Langkah Kompilasi

Pastikan berada di direktori projek_akhir/, kemudian jalankan:

```
$ make
```

Perintah ini mengkompilasi semua file .cpp menjadi binary kmeans_benchmark dengan flag -O2 -std=c++17 -fopenmp.

4.3 Menjalankan Program

Sintaks umum:

```
$ ./kmeans_benchmark <path_gambar> [ukuran_resize]
```

Contoh penggunaan:

```
$ ./kmeans_benchmark sawah6.png 100 # cepat (10.000 piksel)
$ ./kmeans_benchmark sawah6.png 500 # resolusi lebih tinggi
$ make run                          # shortcut (sawah6.png 100)
```

Parameter `ukuran_resize` bersifat opsional (default: 100). Nilai lebih besar menghasilkan segmentasi lebih detail namun waktu eksekusi lebih lama.

4.4 File Kernel OpenCL

File `kernel.cl` harus berada di direktori yang sama dengan binary. File ini berisi fungsi GPU `assign_labels` yang dikompilasi secara runtime oleh driver OpenCL. Jika file tidak ditemukan, program akan keluar dengan pesan error.

5. Input dan Output Program

5.1 Input Program

Input	Tipe	Contoh Nilai	Keterangan
path_gambar	String (path file)	sawah6.png	File gambar RGB (PNG/JPG/dll)
ukuran_resize	Integer (opsional)	100, 500	Lebar & tinggi setelah resize
kernel.cl	File teks OpenCL	kernel.cl	Kode GPU, harus di direktori sama

5.2 Output Program

File output yang dihasilkan:

File Output	Format	Keterangan
result_sequential.png	PNG, RGB	Segmentasi oleh K-Means sequential
result_openmp.png	PNG, RGB	Segmentasi oleh K-Means OpenMP
result_opencl.png	PNG, RGB	Segmentasi oleh K-Means OpenCL

Contoh output konsol:

```
=== BENCHMARK K-MEANS SEGMENTASI CITRA (Mahalanobis) ===
Gambar   : sawah6.png
Ukuran   : 100 x 100 | K: 5 | Max Iterasi: 300

Metode    Waktu (ms) Speedup
Sequential 1254.32  1.00x
```

OpenMP	312.18	4.02x
OpenCL	98.45	12.74x

Setiap piksel dalam gambar output diwarnai sesuai dengan warna centroid cluster yang ditugaskan, menghasilkan visualisasi segmentasi dengan $K = 5$ warna berbeda yang merepresentasikan region-region dominan dalam citra asli.